

Ibis: A purely functional programming language to experiment with categorical memory safety

Charlotte W.

September 4, 2026

Contents

1	Introduction	1
1.1	Syntax Overview	3
1.2	Site Declarations	3
2	Core: The Core Language	4
3	The MLTT based type system	5
4	Category Theory	6
4.1	Yoneda Lemma	6
4.2	Opposite Categories and Presheaves	6
4.3	Restriction Maps	7
4.4	Kan Extensions	7
4.5	Glueing axiom	8
4.6	Geometric Morphisms	8
5	Glossary	9
5.1	Typing Judgments	9

1 Introduction

Ibis is an experimental programming language designed to explore the application of category theory to memory safety, and to answer the question: "can we use category theory to reason about memory safety in a systems language?"

Memory regions in Ibis are treated as open sets in a topological space, forming a presheaf over the category of memory regions $Psh(Mem)$. This allows us to reason about memory safety in a categorical framework, where inclusion morphisms represent safe transformations between memory regions.

1.1 Syntax Overview

1.2 Site Declarations

Site declarations are used to define a topological site in Ibis. Each site declaration consists of a name, a list of covers (open sets), and a list of paths between these functions.

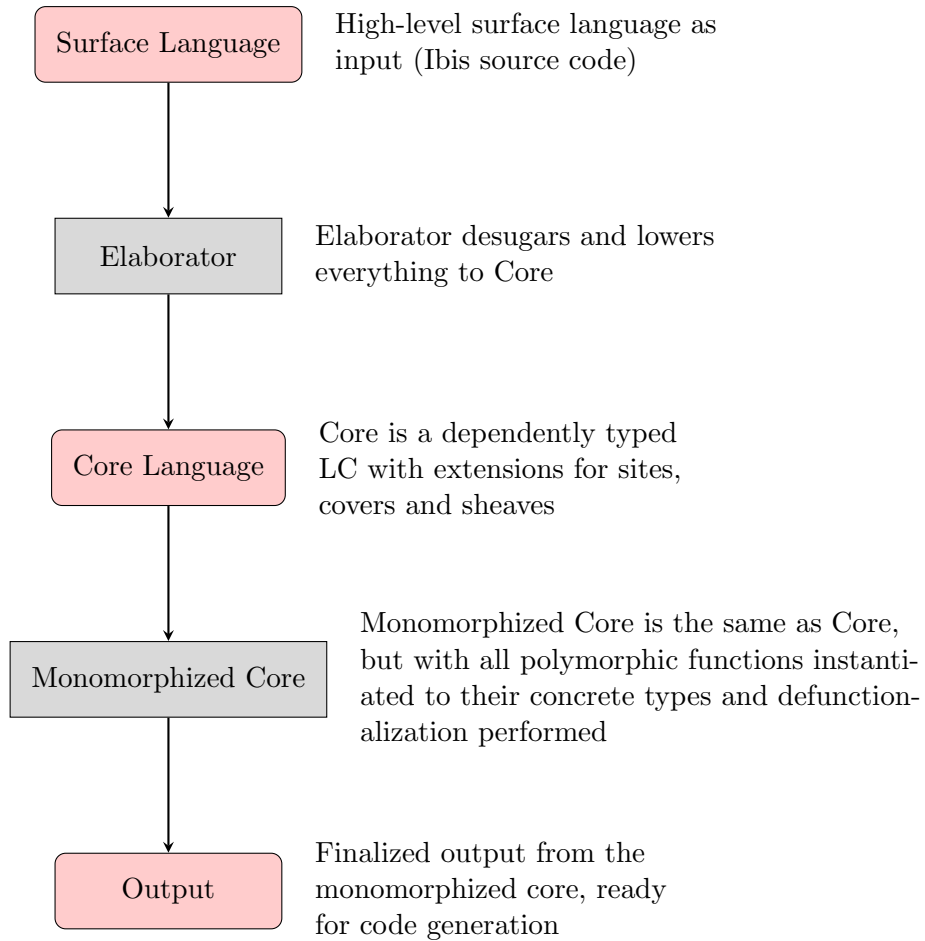
Site declarations inherently map to a cubical type, where the covers are the faces of the cube and the paths are the edges of the cube.

```
1 site MySite where
2   cover @J has [@K, @L]
3
4 -- Paths are automatically generated from site covers
5
6 --  $\sigma$  is the implicit site parameter
7 def alloc_in { $\sigma$  : MySite} (x: Int) : Ptr Int { $\sigma$ } := ...
```

Listing 1: Example of a site declaration in Ibis

2 Core: The Core Language

The core language is Ibis is a dependently typed lambda calculus with extensions for sites, covers and sheaves. Surface is desugared to Core by the elaborator pass, and type-checking, inference, and unification (Miller’s Higher Order Pattern Unification) are performed on Core.



3 The MLTT based type system

The type system of Ibis is based on Martin-Löf Type Theory (MLTT), which provides a foundation for dependently typed programming languages. In Martin-Löf Type Theory, there is no separation between types and terms – types are terms, and terms are types.

We introduce a bidirectional type system, which consists of two judgments: synthesis ($\Gamma \vdash e \Rightarrow A$) and checking ($\Gamma \vdash e \Leftarrow A$). Type synthesis is the process of inferring the type of a term, while type checking is the process of verifying that a term has a specific type.

Martin-Löf Type Theory is a predicative type system rather than impredicative, and organizes types into a hierarchy of universes $\mathcal{U}_0, \mathcal{U}_1, \mathcal{U}_2, \dots$, where $\mathcal{U}_i : \mathcal{U}_{i+1}$. This prevents Girard’s paradox, which arises from systems that allow a type to be a member of itself (i.e., $\mathcal{U} : \mathcal{U}$), leading to a recursive loop in the type checker. Ibis is based on a predicative type system with one exception - Prop; the type of propositions is impredicative, meaning that universal logical quantification over propositions is now well-formed ($\forall x. P(x)$) without bumping universe levels.

$$\begin{array}{c}
\text{VAR} \\
\frac{x : A \in \Gamma}{\Gamma \vdash x \Rightarrow A}
\end{array}
\qquad
\begin{array}{c}
\text{PII-FORM} \\
\frac{\Gamma \vdash A \Leftarrow \mathcal{U} \quad \Gamma, x : A \vdash B \Leftarrow \mathcal{U}}{\Gamma \vdash \Pi(x : A).B \Rightarrow \mathcal{U}}
\end{array}$$

$$\begin{array}{c}
\text{SUM-FORM} \\
\frac{\Gamma \vdash A \Leftarrow \mathcal{U} \quad \Gamma, x : A \vdash B \Leftarrow \mathcal{U}}{\Gamma \vdash \Sigma(x : A).B \Rightarrow \mathcal{U}}
\end{array}
\qquad
\begin{array}{c}
\text{APP} \\
\frac{\Gamma \vdash f \Leftarrow \Pi(x : A).B \quad \Gamma \vdash x \Leftarrow A}{\Gamma \vdash f x \Rightarrow B}
\end{array}$$

$$\begin{array}{c}
\text{FST} \\
\frac{\Gamma \vdash p \Leftarrow \Sigma(x : A).B}{\Gamma \vdash \text{fst}(p) \Rightarrow A}
\end{array}
\qquad
\begin{array}{c}
\text{SND} \\
\frac{\Gamma \vdash p \Leftarrow \Sigma(x : A).B}{\Gamma \vdash \text{snd}(p) \Rightarrow B[\text{fst}(p)/x]}
\end{array}$$

These correspond to the following minimally written Haskell implementation rules: Do note that for brevity, functions have been substituted with their mathematical symbols. A and B represent the domain and co-domain of the function respectively, and $\mathcal{U}$ represents the universe level of types.

- **Var Index:** `synth  $\Gamma$  (Var ix)` looks up the type in Γ at index `ix`.
- **PII-Form:** `synth  $\Gamma$  (PII name A B)` checks A against $\mathcal{U}$, extends Γ with A evaluated to a Value, and checks B against $\mathcal{U}$.

- **Σ -Form:** `synth  $\Gamma$  ( $\Sigma$  name A B)` mirrors the rules of the Π -Form, to synth universe formation for dependent pair types, checking A and B against $\mathcal{U}$.
- **App:** `synth  $\Gamma$  (App rator rand)` synths type of `rator`, verifies it is a Π value type, checks `rand` against A, and applies the closure to `eval rand`.
- **Fst:** `synth  $\Gamma$  (Fst p)` synths type of `p`, verifies it is a Σ value type, and returns A.
- **Snd:** `synth  $\Gamma$  (Snd p)` synths type of `p`, verifies it is a Σ value type, and applies the closure at `eval (Fst p)` to return B.

4 Category Theory

4.1 Yoneda Lemma

The Yoneda lemma is a foundational result in category theory that states that an object in a category is defined entirely by its arrows, and how it relates to other objects in the category.

For any locally small category $\mathcal{C}$, an object $X \in \mathcal{C}$, and a presheaf functor $F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set}$, there is a natural bijection:

$$\text{Nat}(\text{Hom}_{\mathcal{C}}(-, X), F) \cong F(X)$$

In other words, natural transformations from the representable hom-functor $\text{Hom}_{\mathcal{C}}(-, X)$ to F are in a one-to-one correspondence with the elements of the set $F(X)$.

This yields the Yoneda Embedding $\mathbf{y} : \mathcal{C} \rightarrow \mathbf{Set}^{\mathcal{C}^{\text{op}}}$, sending $X \mapsto \text{Hom}_{\mathcal{C}}(-, X)$, which fully and faithfully embeds $\mathcal{C}$ into its presheaf category while preserving its internal structure.

4.2 Opposite Categories and Presheaves

A presheaf $\mathcal{F}$ over a category $\mathcal{C}$ is a contravariant functor $\mathcal{F} : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set}$, where $\mathcal{C}^{\text{op}}$ is the opposite category of $\mathcal{C}$ and $\mathbf{Set}$ is the category of sets, such that for every morphism $f : X \rightarrow Y$ in $\mathcal{C}$, there is a corresponding morphism $f^{\text{op}} : Y \rightarrow X$ in $\mathcal{C}^{\text{op}}$.

4.3 Restriction Maps

Given a valid morphism $f : \sim a \rightarrow \sim b$ between covers $\sim a$ and $\sim b$ the restriction map is defined as:

$$\text{res}_{U,V} : F(\sim a) \longrightarrow F(\sim b)$$

Morphism reachability is treated as a restriction over a presheaf – if no valid morphisms exist between two covers, then the restriction map is undefined and it is mathematically impossible to reach one cover from the other.

4.4 Kan Extensions

Kan extensions are a fundamental concept in category theory that allow us to extend functors along a given functor.

Given a valid morphism $f : \sim a \rightarrow \sim b$ between covers $\sim a$ and $\sim b$, the left Kan extension of a functor F along f is defined as:

$$\text{ext}_{\sim a \rightarrow \sim b} : F(\sim a) \longrightarrow F(\sim b) \quad \text{or} \quad \text{Lan}_f F(\sim b)$$

Definition 4.1 (Left Kan Extension). *Given a functor $F : \mathcal{C} \rightarrow \mathcal{E}$ and a mapping $K : \mathcal{C} \rightarrow \mathcal{D}$, the Left Kan extension of F along K is a functor $\text{Lan}_K F : \mathcal{D} \rightarrow \mathcal{E}$ defined by the colimit for each object d in $\mathcal{D}$, where the colimit is taken over the comma category $(K \downarrow d)$.*

$$(\text{Lan}_K F)(d) = \varinjlim_{(b, K(b) \rightarrow d)} F(b)$$

In other words, to evaluate the left Kan extension of F along K at an object d , we gather all objects b in $\mathcal{C}$ such that there exists a morphism from $K(b)$ to d in $\mathcal{D}$, and then glue them together using a colimit.

Definition 4.2 (Right Kan Extension). *Given a functor $F : \mathcal{C} \rightarrow \mathcal{E}$ and a mapping $K : \mathcal{C} \rightarrow \mathcal{D}$, the Right Kan extension of F along K is a functor $\text{Ran}_K F : \mathcal{D} \rightarrow \mathcal{E}$ defined by the limit for each object d in $\mathcal{D}$, where the limit is taken over the comma category $(d \downarrow K)$.*

$$(\text{Ran}_K F)(d) = \varprojlim_{(d \rightarrow K(b), b)} F(b)$$

4.5 Glueing axiom

For any open cover $\{U_i\}_{i \in I}$ of an open set U , a family of local sections $s_i \in F(U_i)$ is compatible if their restriction maps commute across every pairwise intersection $U_i \cap U_j$ (the union of all pairwise intersections is the entire cover):

$$\rho_{U_i, U_i \cap U_j}(s_i) = \rho_{U_j, U_i \cap U_j}(s_j)$$

If this condition holds, there exists a unique global section $s \in F(U)$ such that s restricts to s_i on each U_i :

$$\rho_{U, U_i}(s) = s_i \quad \forall i \in I$$

In other words: Local data $s_u \in F(U)$ and $s_v \in F(V)$ admit a unique amalgamation $s_{u \cup v} \in F(U \cup V)$ *if and only if* their restriction pullback maps yield identical images in the intersection domain $F(U \cap V)$.

4.6 Geometric Morphisms

A geometric morphism $f : \mathcal{E} \rightarrow \mathcal{F}$ between topoi $\mathcal{E}$ and $\mathcal{F}$ consists of a pair of adjoint functors (f^*, f_*) where $f^* \dashv f_*$:

Geometric morphisms are functorial mappings between topoi that preserve the structure of the topos, including finite limits and colimits.

- The **inverse image functor** $f^* : \mathcal{F} \rightarrow \mathcal{E}$ is the *left adjoint*. It is left-exact (preserves finite limits) and preserves all colimits, used to pull back contexts and sections.
- The **direct image functor** $f_* : \mathcal{E} \rightarrow \mathcal{F}$ is the *right adjoint*. It preserves all limits, used to push forward sections from one site to another.

To calculate the direct image $f_*(P)$ of a presheaf $P \in \mathcal{E}$:

1. Identify the continuous site functor $u : \mathcal{C}_{\mathcal{F}} \rightarrow \mathcal{C}_{\mathcal{E}}$ associated with the geometric morphism f .
2. For each object $V \in \mathcal{C}_{\mathcal{F}}$, evaluate P at its preimage object $u(V) \in \mathcal{C}_{\mathcal{E}}$, yielding $f_*(P)(V) = P(u(V))$.
3. For each target morphism $g : V' \rightarrow V$ in $\mathcal{C}_{\mathcal{F}}$, map it to the corresponding morphism $u(g) : u(V') \rightarrow u(V)$ in $\mathcal{C}_{\mathcal{E}}$.

4. Apply the restriction map $\text{res}_{u(V), u(V')}$ to $P(u(V))$ to obtain $P(u(V'))$, ensuring that the presheaf structure is preserved.

To calculate the inverse image $f^*(Q)$ of a sheaf/presheaf $Q \in \mathcal{F}$:

1. **Construct the Comma Category (Index Graph):** For a source object $U \in \mathcal{C}_{\mathcal{E}}$, form the comma category $(U \downarrow u)$ consisting of pairs (V, ϕ) where $V \in \mathcal{C}_{\mathcal{F}}$ and $\phi : U \rightarrow u(V)$ is a morphism in $\mathcal{C}_{\mathcal{E}}$.
2. **Evaluate the Filtered Colimit (Raw Pullback f_p^*Q):** Compute the raw presheaf pullback at U by taking the colimit of section values over the comma category:

$$(f_p^*Q)(U) = \varinjlim_{(V, \phi) \in (U \downarrow u)} Q(V)$$

This collects all candidate sections $Q(V)$ across target contexts that cover U , identifying sections that agree under restrictions.

3. **First Plus Construction Pass $((f_p^*Q)^+)$:** For each object U and covering sieve $R \in J(U)$, gather matching families of sections in f_p^*Q that agree on overlaps to construct local gluing candidates:

$$(f_p^*Q)^+(U) = \varinjlim_{R \in J(U)} \text{Matching}(R, f_p^*Q)$$

4. **Second Plus Construction Pass $(f^*Q = ((f_p^*Q)^+)^+)$:** Re-apply the plus construction pass to enforce unique local existence and unicity of glued sections, yielding a true sheaf $f^*Q \in \mathcal{E}$.

5 Glossary

5.1 Typing Judgments

$\frac{\text{SYNTHESIS} \quad \Gamma \vdash e \Rightarrow A}{\Gamma \vdash e : A}$	$\frac{\text{CHECKING} \quad \Gamma \vdash e \Leftarrow A}{\Gamma \vdash e : A}$
--	--

Do note that this is to be read as "the top implies the bottom" – the top is the premise and the bottom is the conclusion. We define the premise as a judgment that must be satisfied in order for the conclusion to hold.

- **Synthesis:** The synthesis judgment $\Gamma \vdash e \Rightarrow A$ states that in context Γ , the term e synthesizes (infers) the type A .

- **Checking:** The checking judgment $\Gamma \vdash e \Leftarrow A$ states that in context Γ , the term e checks against the type A .
- **Category:** A category $\mathcal{C}$ consists of a collection of objects, a collection of morphisms (arrows) between these objects, and two operations: composition ($f \circ g$) and identity id_X , satisfying the associativity and identity laws.
- **Site:** A site is a category $\mathcal{C}$ equipped with a Grothendieck topology J , which specifies how to cover objects in $\mathcal{C}$ with families of morphisms. Sites provide the foundation for defining sheaves and presheaves, allowing us to reason about local data and its global behavior.